Large Language Models for Social Science Research

Session 2. Structured Data with an Agent

Hugo Subtil

PhD Workshop, University of Zurich

October 28, 2026

Today's Session Overview (9:00–12:00)

9:00–9:10 Last Week, and What to Keep in Mind (10 min)

9:10–9:30 How I Actually Work (20 min)

- Inspired by Germain Gauthier (Bocconi)

9:30–9:45 Tokens and Models (15 min)

9:45–10:25 Block 1. Specify (40 min), then a 15-min break

- Your analysis plan, with the agent (06)

10:40–11:15 Block 2. Implement (35 min)

- The agent codes, you read the row counts (07)

11:15–11:45 Block 3. Review and Decide (30 min)

- A fresh session reviews, you decide (08, then 05)

11:45–12:00 Debrief and Next Session (15 min)

Slower than last time: every block is *I show* (10 min), *you do*, *we check* (5 min).

Today is structured data. Brought a text corpus? Keep it for Sessions 3 and 4.

Last Week

9:00 – 9:10

What we saw, and what to keep in mind

Last Week, in One Slide

The argument. An AI system now writes economics papers on its own (APE). Generating a paper is almost free; verifying one is not. The bottleneck moved from *doing* to *judging*, and the chain has to end with a human who understands.

The position. Delegate the work. Keep the skill to check it, and the responsibility for it. Three rules:

1. You delegate execution, never understanding.
2. Read before you accept.
3. Stop when you no longer know what it is doing.

The tool. A folder that remembers: `README.md` (what the project is), `CLAUDE.md` (how the agent behaves), `PROJECT_HISTORY.md` (where we stand). Raw data read-only, prompts versioned, every session ends with a commit.

The exercise. Literature notes on your papers, then one note checked against the PDF, looking for the agent's mistake.

What to Keep in Mind

This is still an open debate. How much to delegate, and how much to check, is not settled. Reasonable people draw the line in different places.

Since last week, I asked many colleagues one question:

Where is your limit on review?

What do you read line by line, and what do you let go?

Today, one answer I find convincing, from Germain Gauthier (Bocconi). Not *the* answer: a workable one, which you can adapt.

What you asked for after Session 1: a slower pace, how to save tokens, how much to code yourself. All three are in today's session. Local models (Ollama) come in Session 4.

How I Actually Work

9:10 – 9:30

Inspired by Germain Gauthier (Bocconi)

How I Use AI

- AI is part of pretty much everything I do on my laptop.

Some use cases

- A quick overview of papers I am unfamiliar with
- Writing code, organising project folders, updating results in a paper
- Proof-checking that the code, the paper and the slides are consistent
- Discussing ideas and analyses while I work on a project
- Improving the clarity of my writing and presentations

Pen and paper when I need to fully understand something (theory, statistics), or for mental clarity (a to-do list, planning ahead).

How I Do Not Use AI

The one thing not to do:

"Write me a full paper and a full slide set on this vague idea while I go play tennis."

- **First, current AIs are terrible at this when left unattended.** It is a very iterative process, and you need to be in the loop.
- **Second, you will present the paper and the results.** Being in the loop is how you understand what you are doing.

The journey is almost as important as the destination in research.

AI can still speed things up, and by a lot. Last week's APE is what happens when nobody is in the loop.

The Setup: One Project Folder

Germain's agent is connected to five tools

Dropbox	project files and data
GitHub	code history and repository
Overleaf	paper and slides
HPC cluster	heavy computation
Gmail	co-authors' comments

The project folder

```
my-project/
  overleaf/   local copy of the paper
  code/       code repository
  data/       raw and processed data
  notes/      working notes
```

How it runs

- The agent runs from the project folder.
- He talks to it in the browser, to see PDFs and figures as they are produced.
- The local copy of the paper and the Overleaf project are always in sync.

The same idea as your folder from last week.
You do not need five tools today: one git folder is enough.

Principle 1: You Are in Charge

- Agents overproduce, expand the scope, and overreact to details.
- Do not let them run unattended for long (overnight), unless the task is very well defined: for example, monitoring jobs on a cluster and relaunching the ones that fail.
- Stay hands-on: in the loop, correcting course, making the judgement calls.
- Being hands-on is also how you keep understanding what you are doing, and keep a map of it in your head, even when you no longer write the code.

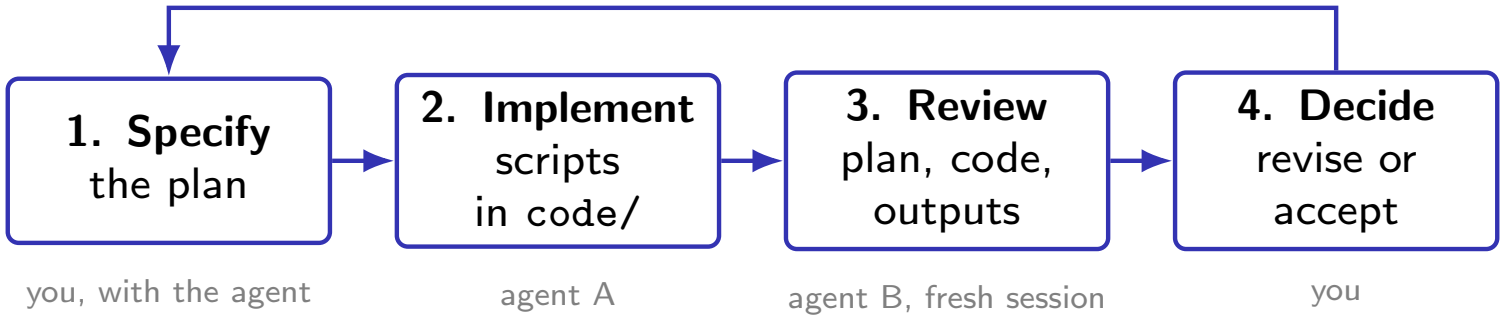
Last week's rule 3, in other words: stop when you no longer know what it is doing.

Principle 2: Accuracy, Not Speed or Cost-Cutting

- Run the models at maximum reasoning. For *everything*.
- A mistake in a paper costs far more than extra tokens or waiting time.
- The difference in intelligence between reasoning levels is large.
- More reasoning does not remove the need for verification.

You asked how to save tokens. This principle says: not here.
At 9:30: what it costs, and where you can save without breaking it.

A Typical Loop



1. **Specify the task in writing.** Sample, equation, estimator, clustering, expected output: as in a detailed pre-analysis plan. Germain writes it in Overleaf; you, in `notes/analysis_plan.md` (06).
2. **A first agent implements it.** It flags ambiguities *before* coding: there is high value in being on the same page (07).
3. **Other agents review it in a fresh context.** They read the plan, the code and the outputs, and list the discrepancies (08).
4. **You decide what to revise or accept.** A reviewer asked to find problems always finds something. Resolve, rerun, repeat until satisfied.

Not exotic: Anthropic's own guide to Claude Code recommends the same steps.

The Typical Failure: Silent Decisions

The failure. The agent takes a consequential decision without asking. Germain's example: dropping some observations. He always asks the reviewers to look for this too. On data: rows lost in a merge, a missing code (−9, 99) read as a value, a duplicate removed, an identifier read as a number (01001 becomes 1001), a library default (inner vs left join).

Last week's question. How would you know that a merge silently dropped a tenth of your observations? Two defences, both in the folder:

1. **Row counts.** Every script prints the number of rows before and after every filter, drop, reshape or merge, and the unmatched keys on each side. A change the plan does not explain stops the agent.
2. **A reviewer that did not write the code.** A fresh session: same model, empty context, no sunk cost. It reads like a referee.

In Block 2, we look for one on the example project.

Two Phases: Prototyping, Then Sharpening

Prototyping

- An exploratory phase: is there something here?
- Do not review the code or the prose.
- Treat the outputs as probable, but provisional.

Sharpening

- There is something: get involved in the coding and the writing.
- Audit the code: the number of rows, results that "look weird".
- Have many independent agents check it.
- Rewrite the paper; iterate on the figures with the AI.
- Take responsibility for the results and the claims.

Why two phases? Checking is expensive, and most ideas die early. Spend the checking on the ones that survive.

Your question: code yourself, or vibe-code? Vibe-code while you prototype. Own the code before anything leaves the folder.

Last week's rule 2, made precise: read before you *build on it*, not before you look at it.

Communication with Co-authors

Keep three distinctions explicit.

Origin	what the AI produced; what I produced
Status	what is exploratory; what is polished
Responsibility	what I have not checked; what I am ready to stand behind

Your co-authors will not look at exploratory and polished results in the same way, and will not make the same comments. AI use and understanding vary across co-authors: be transparent, and take responsibility for your work.

In your folder. From today, the agent labels every output it writes: origin, status, checked. Only you change the last two.

File	Origin	Status	Checked
`tables/t1_descriptives.tex`	agent	exploratory	not yet
`processed/panel.parquet`	agent	polished	HS, counts vs raw
`figures/f1_rn_share.pdf`	agent, then HS	polished	HS, vs table 1

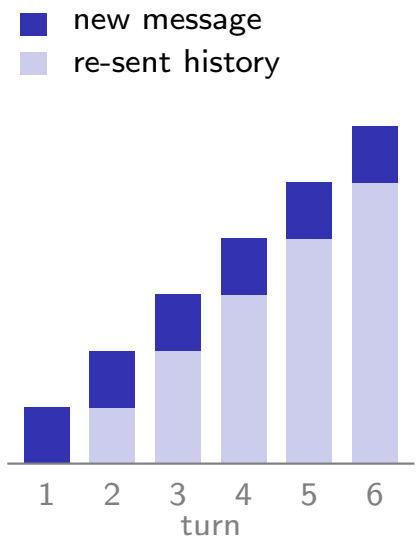
Tokens and Models

9:30 – 9:45

What Principle 2 costs, and where you can save

What Costs Tokens

- **Every message re-sends the whole conversation**: files read, outputs, all previous turns. Turn 40 re-reads turns 1 to 39.
- **The cache discounts what was already sent** while it is warm: about an hour on a subscription, five minutes with an API key. After lunch, full price.
- **Effort multiplies tokens**: more files read, more checks, more thinking. About 7× in one of Anthropic's examples.
- **A bigger model costs more per turn**: Opus several times more than Sonnet.
- **Subagents** spend tokens while you are not typing.



What is sent to the model at each turn

Subscription: tokens are quota. API key: tokens are money.
`/usage` shows where yours went.

Which Model, Which Effort, for Which Task

Model = how capable. Effort = how thorough. It did not *know* enough: a bigger model. It did not *try* hard enough: higher effort.

Default in Claude Code (Pro, Max): Opus 5.5 at `medium`. See it with `/model`; change it with `/effort` (`low`, `medium`, `high`, `xhigh`, `max`).

Task	Model	Effort	Why
Rename, reformat a table, commit message, first sketch	Sonnet	`low`	you check each result anyway
Prototyping: exploratory scripts, quick plots	Sonnet	`medium`	labelled exploratory; speed matters
Writing the analysis plan with you (06)	Opus	`medium`	ambiguity: bigger models handle it better
Code that enters the paper: cleaning, merging, estimation (07)	Opus	`high`	errors hide in cases that are easy to overlook
Independent review (08)	Opus	`xhigh`	the last check before yours
One hard question in a routine session	`ultrathink` in that prompt		deeper reasoning for one turn only

`max`: Germain uses it for everything; Anthropic's documentation warns that on ordinary tasks it tends to overthink. Tight quota: `/model opusplan` (Opus plans, Sonnet codes).

Save on Context, Not on Reasoning

- **One task, one session.** `/clear` between unrelated tasks costs nothing and loses nothing: the project's memory is in the folder (`PROJECT_HISTORY.md`), not in the chat.
- **Watch the first minute.** `Esc` as soon as it drifts; `/rewind` to go back. After two failed corrections: `/clear` and one better prompt, not a third correction.
- **Never paste data.** The agent writes a script that prints summaries (types, counts, missing values); only the summaries enter the context. It also protects confidential data.
- **Point, do not paste.** `@code/03_merge.py`, not the file's content. PDFs: only the pages you need.
- **A short** `CLAUDE.md`: under about 200 lines. Long procedures go in `prompts/`.
- **A long task:** `/compact` with a focus ("keep the merge decisions"). Back after more than an hour: a fresh session costs less.

Save on context. Never on reasoning for what enters the paper.

What You Leave With at Noon

In your folder:

- `notes/analysis_plan.md`, in your words, with its open questions settled
- numbered scripts in `code/` that print their row counts
- a cleaned dataset in `data/processed/` and a first table or figure in `outputs/`
- a review written by a fresh session, and your decision on each point it raised
- an entry in `PROJECT_HISTORY.md` with the labels (origin, status, checked), and a commit

And one thing you will have seen: a decision the agent took on your data without asking. Finding it is Block 3. It is the only step you cannot skip.

Block 1. Specify (9:45–10:25)

I show (10 min). The example project: two raw files, election results and municipal indicators. Prompt 06: the agent inspects the files without changing them and interviews me. I answer; it writes the plan.

You do (25 min)

1. Your data in `data/raw/`, its source in `data/raw/README.md`. No data? The example pack, in the course repository.
2. A new session in your project folder. Paste prompt 06, from the course repository (`template/prompts/`).
3. Answer its questions. What you do not know yet goes to "Open questions". It asks; it does not decide.
4. Read the plan, then let it save `notes/analysis_plan.md`.

We check (5 min). Two plans on screen. What is in their "Open questions"? Which merge has no rule for the rows that do not match?

Block 2. Implement (10:40–11:15)

I show (10 min). `/clear`, then prompt 07. The agent lists what the plan leaves open; I answer. It writes the scripts and prints the row counts. We read them together: where did the rows go?

You do (20 min)

1. `/clear`: the plan is in the folder, not in the chat. Paste prompt 07.
2. Answer its list of ambiguities before it writes any code.
3. Read the row-count trail it gives you, step by step.
4. Rows that change without the plan saying so: paste the step in the shared document.

We check (5 min). The shared document: who lost rows, at which step, and was it in the plan?

Block 3. Review and Decide (11:15–11:45)

I show (10 min). I close the session and open a fresh one. Prompt 08: a reviewer that did not see the code being written. I go through its review and decide, point by point.

You do (15 min)

1. Close the session. Open a new one. Paste prompt 08. Not optional.
2. For each point of the review: fix it, or accept it and write down why.
3. Rerun. Every change in the row counts now follows from the plan.
4. Prompt 05: the entry in `PROJECT_HISTORY.md`, with the labels, and the commit.

We check. That is the debrief.

Debrief

Three screens. An analysis plan. A row-count trail. A review, with the discrepancy it found.

The decisions the agent took alone today. Read from the shared document. The list is the lesson.

One question. The table you would have sent your supervisor an hour ago: which status would it have carried?

Before you leave. Ask the agent to copy two sections of `template/CLAUDE.md` into yours: "Row counts and silent decisions" and "Three labels on every output". Then add one line of your own that would have caught today's error.

Next Session: 4 November 2026

Session 3: Building Datasets from APIs, and Classic Text-as-Data

- Collecting structured and unstructured data through APIs, with the agent writing the collector
- Information extraction without an LLM: dictionaries, embeddings
- Bring one source you would like to collect from: an API, a website, a set of documents
- Keep working in your folder. End every session with prompt 05.

Carry-over question

An API returns 10,000 documents and the agent reports that the collection is complete. How would you know?

References

Gauthier, G. (2026). Using AI in Research Workflows. Slides, Bocconi University, September 2026. pinchofdata.github.io/germaingauthier. The section "How I Actually Work" follows his slides closely.

Anthropic (2026). Best Practices for Claude Code. code.claude.com/docs/en/best-practices.

Anthropic (2026). Model Configuration: Aliases and Effort Levels. code.claude.com/docs/en/model-config.

Anthropic Support (2026). Models and Usage Limits in Claude Code. support.claude.com.

Hallie, L. (2026). Choosing a Claude Model and Effort Level in Claude Code. *Claude Blog*, 7 July 2026.